

Bayou Bombers Throws App

Transferable UI Build Instructions

HTML / CSS / JavaScript graphics and interaction guide



Built from the mockup: coach dashboard, athlete dashboard, module cards, practice builder, team styling.

1. Purpose and Design Intent

This document converts the mockup into transferable front-end build instructions. It is written so a developer can recreate the look and behavior in a self-hosted Python web app using HTML, CSS, and JavaScript. The design should feel like a team-branded command center for throws training, not a generic fitness tracker.

Core app idea

- Coach-controlled system for creating practices, lifts, athlete assignments, modules, and reports.
- Athlete-facing system for viewing Today's Practice, expanding modules, entering results, and submitting performance data.
- Module-based workflow: reusable training blocks are created once and reused to build daily practice plans.
- Event-specific organization for Shotput, Discus, and Javelin.
- Date-driven interface with calendar access for today, past sessions, and future scheduled work.

The mockup communicates a dark aircraft-panel theme with red bomber buttons, tan paper module cards, circular athlete avatars, and a Bayou Bombers logo layer. Build the interface as reusable components so the same visual language can power every page.

2. Graphic Element Inventory

Create these reusable UI elements first. Most screens can then be assembled from the same components.

Element	Where it appears	Recommended implementation
Dark aircraft metal background	All app screens	Use one background image plus CSS overlays. Add rivet lines with pseudo-elements if needed.
Bayou Bombers logo	Coach home, athlete background	Transparent PNG. Large centered image on coach page; faded absolute-position layer on athlete page.
Glossy red buttons	Main navigation, bottom navigation, submit actions	CSS gradients, inset highlights, drop shadow, text shadow. Do not make as images.
Orange button variant	Modules/nav emphasis	Same component as red button with orange gradient.
Dark pill buttons	Secondary athlete controls	Rounded capsule gradient with olive/black tones.
Circular avatars	Event cards, profile areas	img elements with border-radius: 50%, tan border, drop shadow, overlap row.
Event group cards	Coach Today's Practice panel	Rounded dark olive cards with title and avatar row.
Calendar icon button	Coach and athlete screens	Small square button containing a PNG or SVG calendar icon.
Tan module cards	Practice builder and athlete practice list	Reusable CSS card with title, preview list, Expand and Add buttons, optional inputs.
Practice builder panels	Create Practice screen	Two-column layout: setup/assignment on left, module library on right.

3. Asset Checklist

These assets should live in your static image folder. Buttons, cards, forms, panels, and layout should be CSS-driven so they can resize cleanly.

```
/static/images/metal-bg.png
/static/images/bayou-bombers-logo.png
/static/images/calendar-icon.png
/static/images/default-athlete-avatar.png
/static/images/avatar1.png
/static/images/avatar2.png
/static/images/avatar3.png
```

Optional overlays:

```
/static/images/panel-scratch-overlay.png
/static/images/rivet-strip.png
/static/images/bomber-icon-small.png
```

Suggested palette

Bomber Black	Charcoal	Bomber Red	Dark Red	Orange	Vintage Tan	Paper	Olive
#050505	#232323	#C41208	#6F0702	#D97908	#EAD8B8	#DCC89F	#68604B

```
:root {
  --bb-black: #050505;
  --bb-charcoal: #232323;
  --bb-red: #C41208;
  --bb-dark-red: #6F0702;
  --bb-orange: #D97908;
  --bb-tan: #EAD8B8;
  --bb-paper: #DCC89F;
  --bb-olive: #68604B;
  --bb-brown: #4B3520;
  --bb-line: #B99B67;
}
```

4. Global Layout Foundation

Start with a single global screen class. Every page should inherit the metal background, text color, and basic spacing.

```
.app-screen {
  min-height: 100vh;
  position: relative;
  overflow-x: hidden;
  color: var(--bb-tan);
  background-image:
    linear-gradient(rgba(0,0,0,.35), rgba(0,0,0,.75)),
    url("/static/images/metal-bg.png");
  background-size: cover;
  background-position: center;
  font-family: Arial, Helvetica, sans-serif;
}
```

Optional CSS rivet line

```
.rivet-line {
  position: absolute;
  top: 0;
  bottom: 0;
  width: 2px;
  background: rgba(255,255,255,.08);
}
.rivet {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: radial-gradient(circle, #777 0%, #222 70%);
  box-shadow: inset 1px 1px 2px rgba(255,255,255,.3);
}
```

5. Logo Layers

Use the logo in two different ways: as the main focal point on the coach home screen, and as a faded background watermark on the athlete screen.

Main coach logo	Faded background logo
<pre></pre>	<pre></pre>
<pre>.main-logo { width: 240px; max-width: 80%; display: block; margin: 20px auto; filter: drop-shadow(0 0 12px rgba(210,20,10,.8)); } .background-logo { position: absolute; width: 260px; opacity: .25; top: 80px; left: 50%; transform: translateX(-50%); pointer-events: none; }</pre>	

6. Reusable Buttons

The buttons should be CSS components, not image files. The red and orange buttons share the same base class and only change the gradient colors.

```
.btn {
  border: none;
  cursor: pointer;
  font-size: 1rem;
  font-weight: 600;
  color: #fff3dc;
  padding: 10px 16px;
  min-height: 42px;
  text-shadow: 0 1px 2px #000;
  box-shadow: inset 0 2px 2px rgba(255,255,255,.35),
              inset 0 -4px 4px rgba(0,0,0,.45),
              0 4px 6px rgba(0,0,0,.7);
}
.btn-red {
  background: linear-gradient(to bottom, #ef2818 0%, #c41208 45%, #6f0702 100%);
  border-radius: 4px;
}
.btn-orange {
  background: linear-gradient(to bottom, #f6a12c 0%, #d97908 50%, #8a4300 100%);
  border-radius: 4px;
}
.btn:hover { transform: translateY(-1px); filter: brightness(1.1); }
.btn:active { transform: translateY(1px); }
```

Dark capsule button variant

```
.btn-pill {
  border: none;
  border-radius: 999px;
  padding: 9px 18px;
  color: var(--bb-tan);
  background: linear-gradient(to bottom, #68604b 0%, #2d2a21 45%, #050505 100%);
  box-shadow: inset 0 2px 2px rgba(255,255,255,.3),
              inset 0 -3px 4px rgba(0,0,0,.6),
              0 4px 6px rgba(0,0,0,.75);
}
```

7. Avatars, Event Cards, and Calendar Button

Avatars visually show which athletes belong to each event group. Event cards should appear on the coach dashboard inside the Today's Practice panel.

Avatar HTML	Event card HTML
<pre><div class="avatar-row"> </div></pre>	<pre><div class="event-card"> <h3>Shotput</h3> <div class="avatar-row">...</div> </div></pre>

```
.avatar {
  width: 34px;
  height: 34px;
  border-radius: 50%;
  object-fit: cover;
  border: 2px solid var(--bb-tan);
  background: #111;
  box-shadow: 0 2px 5px rgba(0,0,0,.8);
}
.avatar-row { display: flex; align-items: center; gap: 4px; }
.avatar-row .avatar:not(:first-child) { margin-left: -8px; }
.event-card {
  min-height: 58px;
  padding: 8px 12px;
  border-radius: 14px;
  background: linear-gradient(to bottom, rgba(116,109,84,.9), rgba(43,40,30,.95));
  border: 1px solid rgba(230,210,160,.25);
  box-shadow: inset 0 2px 3px rgba(255,255,255,.18), 0 4px 8px rgba(0,0,0,.75);
}
.event-card h3 { margin: 0 0 4px; color: #f5dfb7; font-size: 1rem; }
```

Calendar button

```
<button class="calendar-btn">
  
</button>
```

```
.calendar-btn {
  width: 42px;
  height: 42px;
  border-radius: 8px;
  border: 2px solid var(--bb-tan);
  background: #1a1a1a;
  padding: 4px;
  cursor: pointer;
  box-shadow: 0 3px 6px rgba(0,0,0,.7);
}
.calendar-btn img { width: 100%; height: 100%; object-fit: contain; }
```

8. Module Card Component

Module cards are the most important reusable component. They appear in the coach practice builder and on the athlete Today's Practice page. Each card can show a title, short preview, Expand button, Add button, dropdowns, rep fields, notes, or completion controls.

```
<div class="module-card">
  <div class="module-header">
    <h3>Warm Up A</h3>
    <button class="small-btn expand-btn">Expand</button>
  </div>
  <ul class="module-preview">
    <li>High Knees</li>
    <li>B Skips</li>
    <li>Soldier Walks</li>
  </ul>
  <button class="small-btn add-btn">Add</button>
  <div class="module-details">
    <p>Full warmup details go here.</p>
  </div>
</div>
```

```
.module-card {
  position: relative;
  background: var(--bb-paper);
  color: #241b12;
  border: 2px solid #8f7a4f;
  border-radius: 10px;
  padding: 8px;
  width: 180px;
  box-shadow: inset 0 2px 2px rgba(255,255,255,.45), 0 4px 8px rgba(0,0,0,.6);
}
.module-header { display: flex; justify-content: space-between; align-items: center; }
.module-header h3 { margin: 0; font-size: 1.1rem; color: var(--bb-brown); }
.module-preview { margin: 4px 0; padding-left: 14px; font-size: .75rem; }
.small-btn { font-size: .7rem; padding: 3px 7px; border-radius: 3px; border: 1px solid #5b3a1a; cursor: pointer; }
.expand-btn { background: #f29931; color: #2a1404; }
.add-btn { background: #e6e4b5; color: #173b13; }
.module-details { display: none; margin-top: 8px; padding: 6px; background: rgba(255,255,255,.25); border-radius: 6px; }
.module-card.expanded .module-details { display: block; }
```

Expand/collapse behavior

```
document.querySelectorAll(".expand-btn").forEach(button => {
  button.addEventListener("click", event => {
    const card = event.target.closest(".module-card");
    card.classList.toggle("expanded");
    event.target.textContent = card.classList.contains("expanded") ? "Collapse" :
      "Expand";
  });
});
```

9. Coach Home Screen

The coach home screen functions as the command dashboard. The left side holds primary actions. The right side summarizes Today's Practice by event group.

```
<section class="coach-home app-screen">
  
  <div class="home-grid">
    <nav class="main-actions">
      <button class="btn btn-red">Create Practice</button>
      <button class="btn btn-red">Create Lift</button>
      <button class="btn btn-red">Reports</button>
      <button class="btn btn-red">Athletes</button>
      <button class="btn btn-orange">Modules</button>
    </nav>
    <aside class="today-panel">
      <button class="calendar-btn"></button>
      <div class="event-card"><h3>Shotput</h3><div class="avatar-
        row">...</div></div>
      <div class="event-card"><h3>Discus</h3><div class="avatar-
        row">...</div></div>
      <div class="event-card"><h3>Javelin</h3><div class="avatar-
        row">...</div></div>
    </aside>
  </div>
</section>
```

```
.home-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 24px;
  max-width: 720px;
  margin: 0 auto;
  padding: 20px;
}
.main-actions, .today-panel { display: flex; flex-direction: column; gap: 10px; }
.today-panel { position: relative; }
```


10. Athlete Today's Practice Screen

The athlete screen should be simple. The athlete logs in, sees assigned modules for the current day, expands details, enters results, checks completion, and submits.

```
<section class="athlete-home app-screen">
  
  <header class="athlete-header">
    <h1>Today's Practice</h1>
    <button class="calendar-btn"></button>
  </header>
  <main class="athlete-practice-list">
    <div class="module-card athlete-module">...</div>
    <div class="module-card athlete-module">...</div>
  </main>
  <nav class="athlete-bottom-nav">
    <button class="btn btn-red">Practice</button>
    <button class="btn btn-red">Lift</button>
    <button class="btn btn-red">Meet Data</button>
    <button class="btn btn-red">Progress</button>
    <button class="btn btn-orange">Profile</button>
  </nav>
</section>
```

```
.athlete-home { position: relative; overflow: hidden; padding: 16px; }
.athlete-header {
  position: relative;
  z-index: 2;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.athlete-header h1 { color: var(--bb-tan); text-shadow: 0 2px 4px #000; }
.athlete-practice-list {
  position: relative;
  z-index: 2;
  display: flex;
  flex-direction: column;
  gap: 12px;
  max-width: 320px;
}
.athlete-module { width: 100%; }
.athlete-bottom-nav {
  position: relative;
  z-index: 2;
  margin-top: 24px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  max-width: 220px;
}
```

Athlete input fields

```
<label>Variation
  <select>
    <option>Power Position</option>
    <option>Step-in</option>
    <option>Full Throw</option>
  </select>
</label>
<label>Reps
  <input type="number">
</label>
<label>Completed?
  <input type="checkbox">
</label>
```

```

input, select, textarea {
  background: var(--bb-tan);
  color: #21170d;
  border: 2px solid #8f7a4f;
  border-radius: 3px;
  padding: 5px;
  font-family: inherit;
}
label { display: block; font-size: .85rem; color: #2d2114; margin-bottom: 4px; }
.field-label { color: var(--bb-tan); text-shadow: 0 1px 3px #000; }

```

11. Create Practice Screen

The Create Practice page is the coach workflow for assigning training. It should contain date selection, athlete/event selection, a practice build area, a module library, and a Create button.

```

<section class="create-practice app-screen">
  <div class="practice-builder">
    <div class="builder-left">
      <input type="date" class="date-input">
      <label class="field-label">Select Athletes</label>
      <select class="select-input">
        <option>--select--</option>
        <option>All Athletes</option>
        <option>Shotput Group</option>
        <option>Discus Group</option>
        <option>Javelin Group</option>
      </select>
      <div class="event-select-box">
        <label>Shotput <input type="checkbox" value="shotput"></label>
        <label>Discus <input type="checkbox" value="discus"></label>
        <label>Javelin <input type="checkbox" value="javelin"></label>
      </div>
      <div class="practice-drop-zone" id="practiceDropZone"></div>
      <button class="btn btn-red create-btn">Create</button>
    </div>
    <div class="builder-right">
      <h1>Modules</h1>
      <div class="module-list">...</div>
    </div>
  </div>
</section>

```

```

.practice-builder {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 18px;
  max-width: 760px;
  margin: 0 auto;
  padding: 20px;
}
.builder-left, .builder-right { background: rgba(0,0,0,.25); border-radius: 14px;
  padding: 14px; }
.date-input, .select-input { width: 100%; margin-bottom: 8px; padding: 6px;
  background: var(--bb-tan); border: 2px solid #8f7a4f; color: #241b12; }
.event-select-box { background: var(--bb-tan); color: #241b12; border: 2px solid
  #8f7a4f; padding: 8px; margin-bottom: 12px; }
.event-select-box label { display: flex; justify-content: space-between; font-
  size: 1.1rem; margin-bottom: 4px; }
.practice-drop-zone { min-height: 130px; background: rgba(220,200,159,.85);
  border: 3px solid #9d875d; border-radius: 14px; padding: 10px; color: #2a1e11; }
.builder-right h1 { text-align: center; color: var(--bb-tan); text-shadow: 0 2px
  4px #000; }
.module-list { display: flex; flex-direction: column; gap: 10px; }

```

12. JavaScript: Add Modules to a Practice

When the coach clicks Add on a module card, create a copy inside the practice build area. This is the front-end behavior; the final Create button should send the selected date, athletes, events, and module IDs to the Python backend.

```
const modules = [
  { id: 1, title: "Warm Up A", items: ["High Knees", "B Skips", "Soldier Walks"] },
  { id: 2, title: "Stand Throws", items: ["Variations", "Reps"] }
];

function addModuleToPractice(moduleId) {
  const module = modules.find(m => m.id === moduleId);
  const dropZone = document.querySelector("#practiceDropZone");

  const addedModule = document.createElement("div");
  addedModule.className = "added-module";
  addedModule.innerHTML = `
    <strong>${module.title}</strong>
    <ul>${module.items.map(item => `<li>${item}</li>`).join("")}</ul>
    <button class="remove-module">Remove</button>
  `;

  dropZone.appendChild(addedModule);
  addedModule.querySelector(".remove-module").addEventListener("click", () => {
    addedModule.remove();
  });
}
```

```
.added-module {
  background: #f0dfbc;
  border: 2px solid #8f7a4f;
  border-radius: 8px;
  padding: 8px;
  margin-bottom: 8px;
}

.remove-module {
  background: #8b0b05;
  color: white;
  border: none;
  padding: 4px 8px;
  cursor: pointer;
}
```

13. Screen Interaction Map

Area	Interaction	Expected result
Coach home	Click Create Practice	Navigate to or render the Create Practice screen.
Coach home	Click Create Lift	Open the lift builder for assigned weight training.
Coach home	Click Reports	Open performance, practice, lift, and meet reports.
Coach home	Click Athletes	Open athlete management and profile pages.
Coach home	Click Modules	Open module library and module creation tools.
Coach home	Click event card	Open that event's practice assignment for today.
Create Practice	Select date	Attach the practice plan to a calendar date.
Create Practice	Select athletes/event	Define who receives the practice.
Create Practice	Click Expand	Preview module details without adding it yet.
Create Practice	Click Add	Append module to the practice plan.
Create Practice	Click Create	Save assignments and make them visible to selected athletes.
Athlete home	Click Expand	Show full module instructions and data fields.
Athlete home	Submit results	Save reps, completion, notes, lift data, or meet data to the backend.

14. Recommended File Structure

```

project/
  app.py
  templates/
    base.html
    coach_home.html
    athlete_home.html
    create_practice.html
    create_lift.html
    modules.html
    athletes.html
    reports.html
  static/
    css/
      styles.css
    js/
      app.js
      practice-builder.js
      modules.js
    images/
      metal-bg.png
      bayou-bombers-logo.png
      calendar-icon.png
      default-athlete-avatar.png

```

15. Backend Data Objects to Support the UI

The mockup is visual, but these data objects are implied by the workflow and should be planned in the Python backend.

Object	Purpose
Users	Authentication and role: coach or athlete.
Athletes	Profile info, event groups, avatars, PRs, lift maxes, notes.
Events	Shotput, Discus, Javelin categories.
Modules	Reusable practice blocks with exercises, reps, variations, notes, and event compatibility.

Object	Purpose
Practice Plans	Date-based plans built from modules.
Practice Assignments	Links practices to athletes and event groups.
Practice Results	Athlete completion, reps, notes, and practice data.
Lift Sessions	Assigned lifting workouts with sets, reps, percentages, and burnout sets.
Lift Results	Weights used, reps completed, projected maxes, and suggested max increase flags.
Meet Performances	Athlete-entered event, distance in feet, date, and location.

16. Implementation Workflow

Build the interface in this order so the app becomes consistent quickly and each later screen reuses existing pieces.

- 1 Create global CSS variables and base .app-screen background.
- 2 Add the metal background image and logo assets.
- 3 Build red, orange, and pill button classes.
- 4 Build avatar rows and event cards.
- 5 Build module cards with Expand and Add behavior.
- 6 Build the coach home screen layout.
- 7 Build the Create Practice screen and module drop zone.
- 8 Build the athlete Today's Practice screen.
- 9 Connect JavaScript actions to backend routes.
- 10 Add forms for meet performance and lift result submission.
- 11 Create reports from practice, lift, and meet result data.
- 12 Test responsiveness on phone, tablet, and desktop widths.

17. Responsive Layout Rules

The mockup is narrow and mobile-friendly, but the self-hosted web app should also work on desktop. Use max-width containers so the layout keeps the designed shape instead of stretching too wide.

```
@media (max-width: 700px) {  
  .home-grid,  
  .practice-builder {  
    grid-template-columns: 1fr;  
  }  
  .main-logo { width: 210px; }  
  .module-card { width: 100%; }  
  .athlete-practice-list,  
  .athlete-bottom-nav { max-width: 100%; }  
}  
  
@media (min-width: 1000px) {  
  .home-grid,  
  .practice-builder {  
    max-width: 880px;  
  }  
}
```

18. Developer Handoff Notes

- Keep the theme image-based only for the background and logos. Build controls with CSS.
- Make every UI piece reusable: .btn, .module-card, .event-card, .avatar-row, .calendar-btn.
- Do not hard-code athlete names or modules in final production. Render them from backend data.
- The coach creates and assigns. The athlete views and submits. Keep permissions strict.
- Module templates should stay reusable; daily assigned modules may be customized without permanently changing the template.
- The calendar should drive practice, lift, and meet data views by date.
- Reports should be generated from athlete submissions, not manually maintained.
- For lifting, support projected max calculations and a suggested max increase flag based on burnout set performance.

Final goal: a branded, date-driven throws training web app where the coach builds workouts from reusable modules and athletes receive a simple daily execution screen.